AUCA | BIG DATA ANALYTICS | FINAL PROJECT

TECHNICAL REPORT

Distributed Multi-Model Analytics for E-commerce Data

MongoDB (document) + HBase (wide-column) + Apache Spark (distributed processing)

STUDENT	**DUSENGIMANA Victor**
STUDENT ID	101372
COURSE	Big Data Analytics - Final Project
DATASET	Synthetic e-commerce data: 1,000 users, 500 products, 40,000 sessions, 10,000 transactions (90 days)

TECHNOLOGIES

MongoDB **HBase** **Apache Spark** **Matplotlib / Seaborn**

SOURCE CODE REPOSITORY (GitHub)

All of the code, scripts and results for this project:

https://github.com/dusengimanagusa-acc/Victor-Big-Data-Analytics-Final-Exam

Table of Contents

1 Introduction and system architecture 3
2 Data modelling (Part 1) 3
2.1 MongoDB document model 3
2.2 HBase wide-column model 5
2.3 Why some data is in MongoDB and some in HBase 5
3 Spark processing pipeline (Part 2) 6
4 Integrated analytics (Part 3) 9
5 Visualisations and business insights (Part 4) 10
6 Scalability considerations 12
7 Technology selection summary 12
8 Limitations and future work 12
Appendix A - Console captures of the pipeline 14
Appendix B - Further screenshots from my own run 18

List of Figures

Figure 1 - System architecture 3
Figure 2 - Daily revenue with 7-day moving average 10
Figure 3 - Top 10 categories: revenue vs. estimated profit 10
Figure 4 - Customer segmentation: CLV tiers and purchase-frequency segments 11
Figure 5 - Conversion funnel (HBase sessions reconciled with MongoDB orders) 11
Figure 6 - Cohort analysis: revenue per user by registration and purchase month 11
Figure 7 - Session-to-order conversion by referrer and by device 11

List of Screenshots

Screenshots 1-2 - MongoDB aggregation pipelines A-D; HBase scans 6
Screenshot 3 - Spark cleaning job and data-quality report 7
Screenshot 4 - Spark SQL queries 1-3 8
Screenshot 5 - Integrated query 1: CLV tiers, correlations, top customers 9
Screenshots A1-A7 - Appendix A: console captures and independent verification 14-17
Screenshots B1-B11 - Appendix B: further captures incl. Spark web UI 18-23

Distributed Multi-Model Analytics for E-commerce Data

MongoDB (document) + HBase (wide-column) + Apache Spark (distributed processing)

AUCA Big Data Analytics - Final Project - Technical Report | DUSENGIMANA Victor (ID 101372)

At a glance. I ran everything at 0.02 of the full specification: 1,000 users, 500 products, 40,000 sessions and 10,000 transactions spread over 90 days.

1. Introduction and system architecture

I set out to answer a handful of business questions about a synthetic e-commerce shop, and I tried to let each technology do the job it is best at. **MongoDB** holds the entity data (catalog, customers, orders), which gets read as whole documents and sliced along many different dimensions. **HBase** takes the session event stream, which is high-volume and append-only, with a row key built for time-ordered access per user. **Apache Spark** does the heavy lifting: full scans, joins and algorithms such as co-occurrence, cohorts and customer lifetime value. It is also where the two stores meet, because the integration job joins profile data from MongoDB with session events from HBase. Figure 1 shows how the pieces fit together, and section 7 explains why I chose each one.

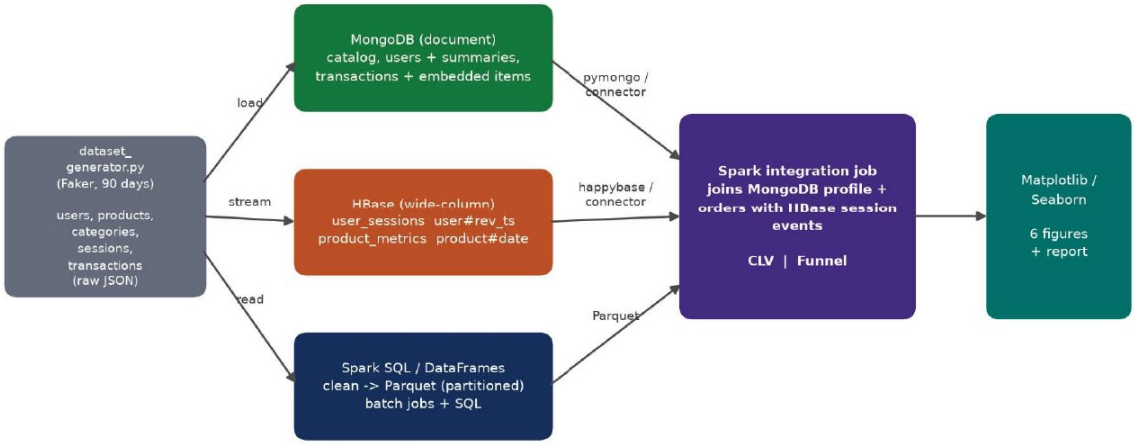


Figure 1 - System architecture. Raw JSON is loaded into MongoDB and HBase, Spark processes it, and the integration job joins the two stores.

2. Data modelling (Part 1)

2.1 MongoDB document model

I designed the model around three things I knew I would need to do: (i) revenue and popularity analytics over orders, (ii) customer profile and segmentation reads, and (iii) product look-ups together with their category. My rule of thumb was simple. If data is always read together and stays small, I **embed** it. If it is shared or queried on its own, I **reference or denormalise** it.

Collection	Design decision	Justification
categories	Subcategories (with profit_margin) are embedded as an array.	They are few, always read together, and a whole category hierarchy fits in one document.
products	price_history is embedded; category and subcategory name/margin are **denormalised** into the product.	Price history is only ever read with its product, and denormalising saves a $lookup on every product listing. *The generator does not emit a subcategory_id, so I assign one deterministically (product number mod the number of subcategories).*
transactions	Line items are embedded, and every item also carries category_id and profit_margin. The timestamp is stored as a real Date (field ts).	An order is written and read together with its items. Revenue and profit by category then need no join, and a real Date type lets me group with $dateToString and run range queries on the ts index.
users	Profile plus an embedded, pre-computed **purchase_summary** (completed/shipped/delivered orders only, the same definition as revenue) and an **engagement_summary** (computed pattern).	Profile pages and segmentation hit a single document, and $bucket over the summary fields is a one-collection pass. The trade-off is that the summaries are redundant, so a batch job or change stream has to keep them fresh.

Indexes (chosen to match the pipelines): transactions(ts), (user_id, ts), (items.product_id); products(category.category_id); users(purchase_summary.total_orders), (geo_data.state). **I deliberately keep sessions out of MongoDB**: they are large, write-heavy time-series documents (up to 16 page views each), which suit HBase and Spark far better.

Here is an example transaction document (a real record from the generated data, after the loading transformations):

```
{
 "_id": "txn_187f1fcf4064",
 "user_id": "user_000724",
 "ts": "ISODate(2026-08-25T19:09:30)",
 "items": [
  {"product_id": "prod_00032", "quantity": 3, "unit_price": 144.47, "subtotal": 433.41,
   "category_id": "cat_008", "profit_margin": 0.36},
  {"product_id": "prod_00184", "quantity": 3, "unit_price": 166.18, "subtotal": 498.54,
   "category_id": "cat_020", "profit_margin": 0.26}
 ],
 "subtotal": 1339.45,
 "discount": 0,
 "total": 1339.45,
 "status": "completed"
}
```

I implemented **four aggregation pipelines** in `mongodb/02_aggregations.py`. **A** is revenue and estimated profit by category and month (`$match, $unwind, $group` with `$dateToString`, `$lookup`). **B** is the top 20 products by revenue, with category names. **C** segments users by purchase frequency with `$bucket`, using boundaries taken from the *quintiles of the real order-count distribution* so the segments stay meaningful at any scale. **D** is revenue by state and payment method (`$lookup` to users).

2.2 HBase wide-column model

Table / key	Column families	Why
user_sessions row key = user_id#(9999999999999 - start_epoch_ms)	**meta**: session attributes, device_*, geo_* **pv**: one sparse column per page view (pv:000, pv:001...) **cart**: cart:<product_id> = qty|price	A user's sessions sit next to each other, **newest first** (reverse timestamp), so "latest N sessions" is one short prefix scan and a date range is one contiguous range scan. user_id leads the key, which spreads writes (no hotspot), and the table is pre-split. I split the families by access pattern: a metadata query never reads the heavy pv family. I kept it to 3 families (each family is a separate file/memstore).
product_metrics row key = product_id#yyyymmdd	**m**: views, view_seconds, cart_adds, cart_units, purchase_sessions, purchased_units	A product's daily history is contiguous, so time-range scans (view-to-purchase trends) are cheap. It is sparse: a product-day with no activity simply has no row.

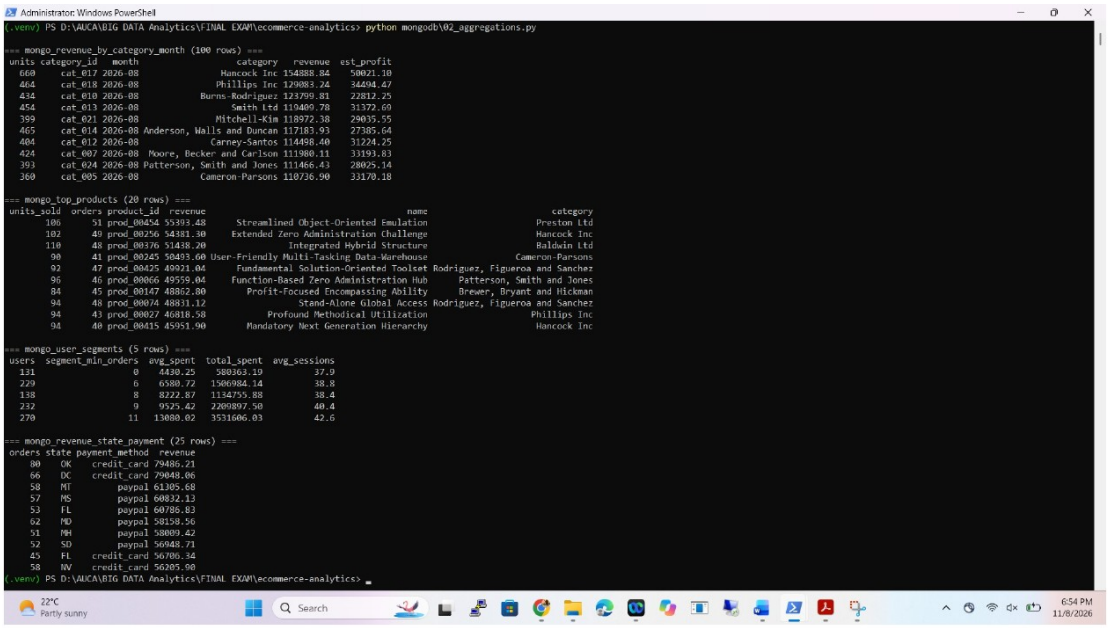
An example HBase row (a real session from the generated data):

```
ROW KEY  user_000872#8212146851999
   meta:start_time = 2026-08-27T17:52:28
   meta:duration_seconds = 2017
   meta:device_type = desktop
   meta:conversion_status = abandoned
   pv:000 = 2026-08-27T17:52:28|home|||34
   cart:prod_00154 = 2|594.15
```

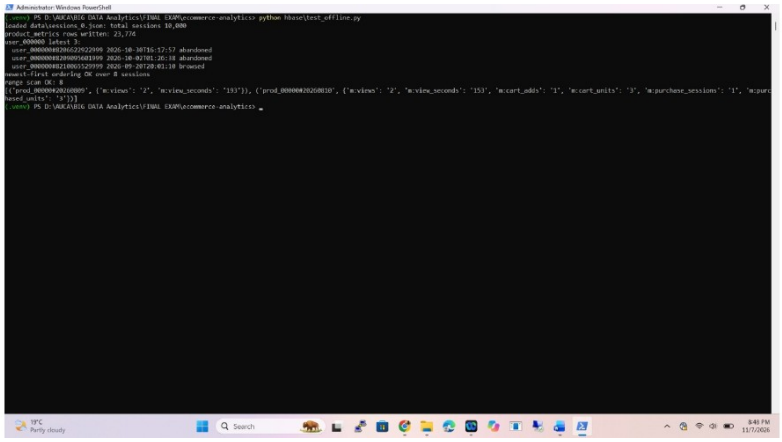
2.3 Why some data is in MongoDB and some in HBase

Need	Best fit	Reason
Ad-hoc, multi-dimensional aggregation over orders and customers	MongoDB	Aggregation framework, secondary indexes, nested documents, and no sorted-key constraint.
Fetching one user's recent activity in milliseconds; time-range scans over events	HBase	Sorted row keys, prefix/range scans, wide sparse rows, very high write throughput and linear scale-out.
Heavy joins, iterative algorithms, full scans of everything	Spark	Distributed in-memory processing; SQL and DataFrames; it can read any of the stores.

HBase has no secondary indexes or joins, so any question the row key cannot answer goes to Spark. MongoDB, on the other hand, is not built for billions of tiny events. That is why order data lives in MongoDB and click-stream data lives in HBase.



Screenshot 1 - MongoDB (real server): aggregation pipelines A-D, including A (revenue by category and month).



Screenshot 2 - HBase: loaded subset, newest-first scan of one user's sessions, range scan and product_metrics row. Offline test harness.

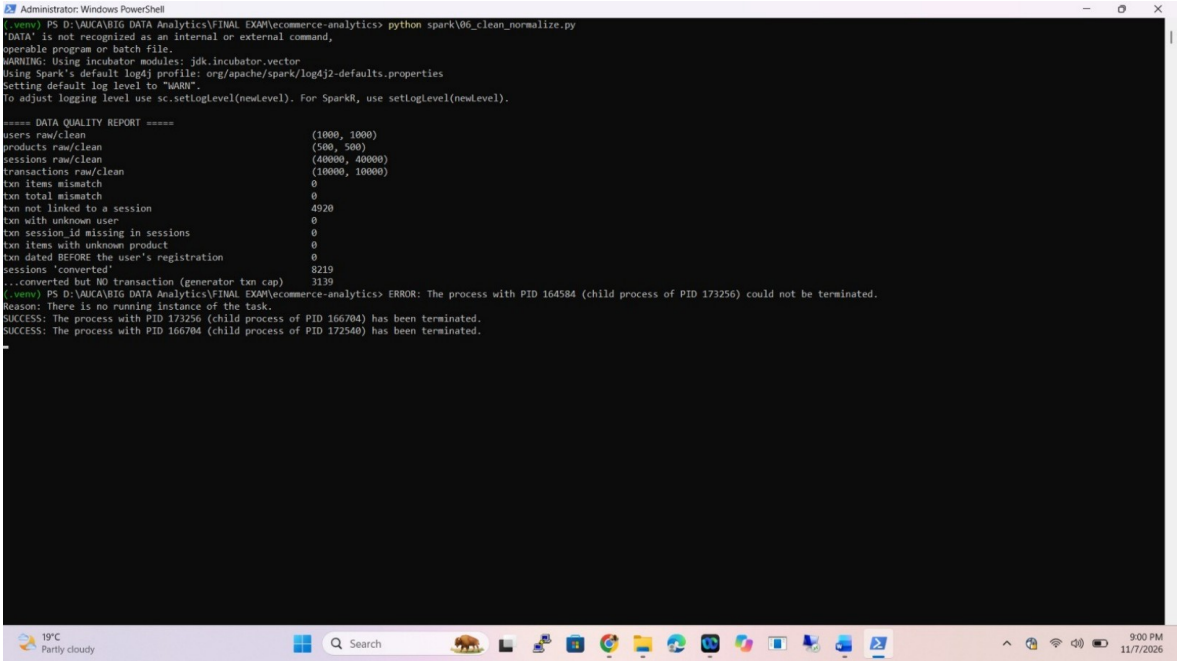
3. Spark processing pipeline (Part 2)

Pipeline. Raw JSON (one JSON array per file, so I read it with `multiLine=true`) → explicit schemas (no inference pass over gigabytes) → a cleaning and normalisation job → partitioned Parquet (sessions by date, transactions by month) → batch analytics and Spark SQL.

Cleaning and normalisation (`06_clean_normalize.py`). I parsed timestamps from ISO strings, removed duplicates on the primary keys, standardised text (state upper-case, device and referrer lower-case) and dropped invalid rows (negative durations, non-positive prices). I exploded the category hierarchy into a flat dimension, recomputed the transaction totals and compared them with the stored values, and checked referential integrity with left-anti joins. Here is the data-quality report:

Check	Result	Check	Result
users raw/clean	(1000, 1000)	txn with unknown user	0
products raw/clean	(500, 500)	txn session_id missing in sessions	0
sessions raw/clean	(40000, 40000)	txn items with unknown product	0
transactions raw/clean	(10000, 10000)	txn dated BEFORE the user's registration	0
txn items mismatch	0	sessions 'converted'	8219
txn total mismatch	0	...converted but NO transaction (generator txn cap)	3139
txn not linked to a session	4920		

Findings. Everything stored is consistent: there are 0 mismatches, every reference resolves, and no transaction predates its user's registration. Two quirks of the synthetic data are worth calling out. First, 4,920 of the 10,000 transactions (49%) have no session link by design, because the generator creates "extra" orders. Second, 3,139 of the 8,219 sessions flagged "converted" (38%) have *no* transaction. I traced that back to the generator: once it reaches the transaction cap it stops creating transactions, but sessions keep getting flagged as converted. For the funnel reconciliation I therefore use the first part of the sessions (the HBase subset), where the two systems agree.



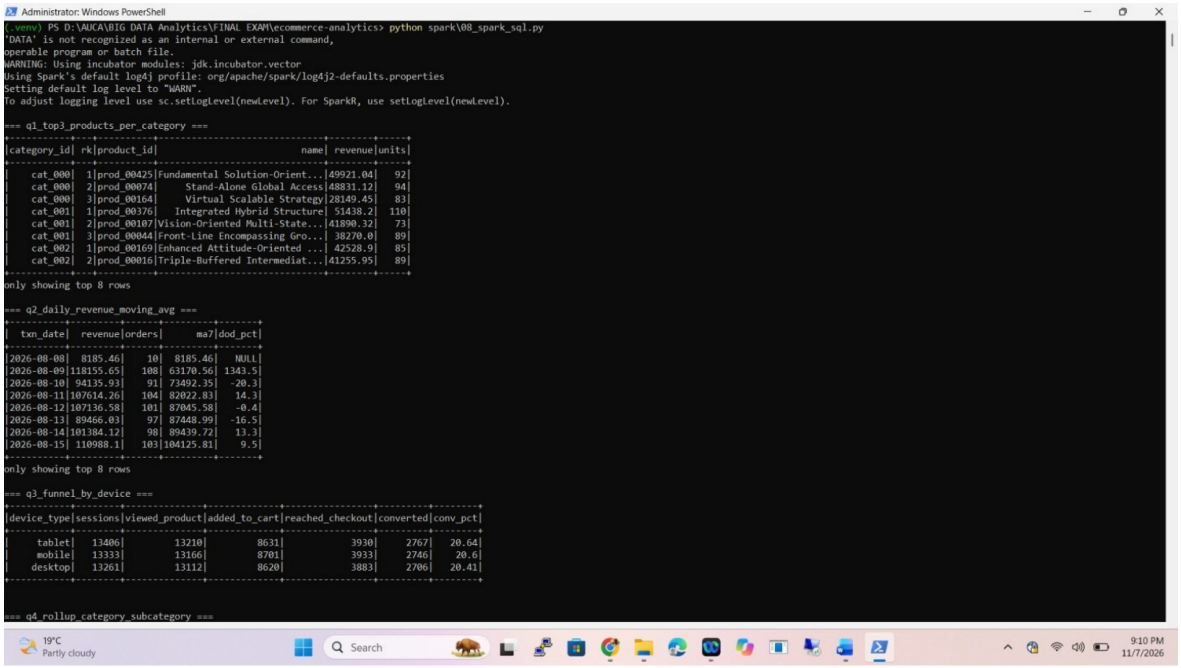
Screenshot 3 - Spark cleaning job and data-quality report.

Batch analytics (`07_batch_analytics.py`). I built three things here. (a) *Frequently bought together* is a self-join of basket items with a.id < b.id (so each unordered pair appears once), with support, confidence and lift. (b) *Frequently viewed together* comes from the sessions. (c) The *cohort analysis* groups users by registration month and measures revenue per user in each purchase month. The top basket pair is prod_00037 + prod_00205 (4 baskets, lift 11.9). Lift values are noisy because the pair counts are small.

What I did to keep it fast. Parquet (columnar) with partition pruning, column pruning, a **broadcast join** of the small product dimension, de-duplicated pairs in the self-join, `cache()` for DataFrames I reuse several times, shuffle partitions tuned to the data size (`SHUFFLE_PARTS`), and explicit schemas instead of inference.

Spark SQL (`08_spark_sql.py`). There are six queries: top-3 products per category (a CTE with a ROW_NUMBER window), a 7-day moving average with day-over-day growth (window frames, LAG), a device funnel (using the higher-order function `exists` on the nested page_views array), revenue/profit ROLLUP by category and subcategory, and two queries on views (mongo_users, mongo_purchase_summary, hbase_sessions) that stand in for data loaded through the MongoDB and HBase connectors. The SQL logic is identical either way. Here is query 1:

```
q1_top3_products_per_category
WITH lines AS (
  SELECT t.transaction_id, i.product_id, i.quantity, i.subtotal
  FROM transactions t LATERAL VIEW explode(t.items) x AS i
  WHERE t.status IN ('completed','shipped','delivered')
), prod_rev AS (
  SELECT p.category_id, p.product_id, p.name, SUM(l.subtotal) AS revenue, SUM(l.quantity) AS units
  FROM lines l JOIN products p ON p.product_id = l.product_id
  GROUP BY p.category_id, p.product_id, p.name
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY revenue DESC) AS rk
  FROM prod_rev
)
SELECT category_id, rk, product_id, name, ROUND(revenue,2) AS revenue, units
FROM ranked WHERE rk <= 3 ORDER BY category_id, rk;
```



Screenshot 4 - Spark SQL queries 1 (top products per category), 2 (moving average) and 3 (device funnel).

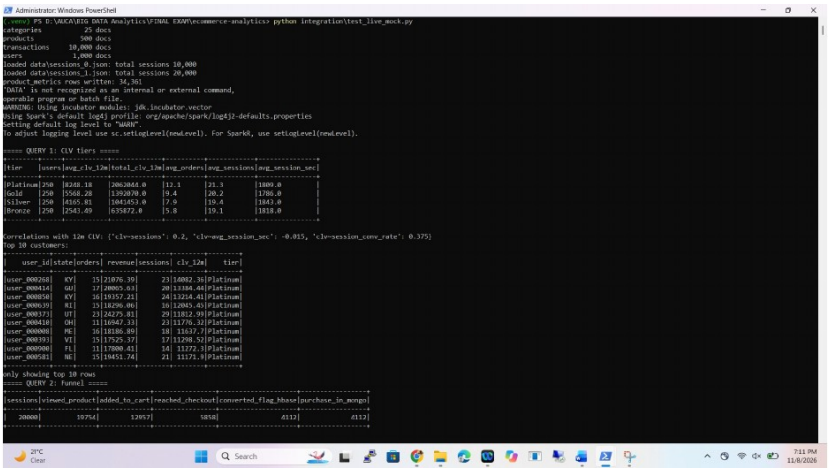
Screenshot 4 – Spark SQL queries 1 (top products per category), 2 (moving average) and 3 (device funnel).

4. Integrated analytics (Part 3)

Query 1 - Customer Lifetime Value (CLV). *Business question:* which customers will be worth the most over the next 12 months, and does browsing engagement predict that? *Sources:* MongoDB (profiles, plus a per-user revenue/profit roll-up computed **inside** MongoDB so only one small row per user leaves the database), HBase (session count, average duration and conversion rate per user), and Spark (the join, CLV, NTILE tiers and correlations). *Formula:* monthly net profit = (gross item profit − discounts) / 3 observed months; CLV_{12m} = monthly net profit × $(1 + r + ... + r^{11})$, with an **assumed** monthly retention of r = 0.90. It is an assumption rather than something estimated from the data, because a 90-day window cannot reveal churn.

Query 2 - Funnel conversion. *Business question:* where do shoppers drop out between the first product view and a confirmed purchase, and does that differ by device or referrer? *Sources:* HBase sessions (view → cart → checkout, device, referrer) and MongoDB transactions (the system of record for "order exists"), joined in Spark on session_id. *Method:* the funnel is **strictly nested**, so a session only counts at a stage if it also passed every earlier stage (view, cart, checkout, converted, order exists). That matters because the generator's page logs are noisy. For example, some sessions with an empty cart still contain a "checkout" page, so counting each stage independently would overstate the later stages.

Workflow. (1) pymongo runs two aggregations in MongoDB. (2) A happybase column-restricted scan reads the HBase sessions and derives the stage flags. (3) Both results become Spark DataFrames. (4) Spark joins on user_id and session_id, then computes CLV, NTILE tiers, correlations and funnel counts. (5) The results are exported to CSV for plotting. **Performance notes:** aggregation is pushed down to MongoDB, and HBase scans are restricted to the column families I need. At scale I would replace the single-threaded Python readers with the MongoDB Spark Connector and the Spark-HBase connector, so reads run in parallel across executors. The same code can also run from Parquet files (`--source files`), and I checked that live mode and file mode give identical results. **Verification:** `tools/verify_results.py` recomputes the headline numbers (revenue totals, top product, funnel stages, CLV of sample customers, the cohort table and the top product pair) from the raw JSON in plain Python and compares them with the MongoDB, Spark and integration outputs. All of the checks pass (see Appendix A7).



Screenshot 5 - Integrated query 1: CLV tiers, correlations and top customers. Offline test harness for the two stores.

5. Visualisations and business insights (Part 4)

Across completed, shipped and delivered orders, revenue comes to **$9,196,373** and estimated profit to **$2,378,104** (a 25.9% margin). The highest-revenue category is Hancock Inc ($540,051, 5.9% of revenue), and the best product is Streamlined Object-Oriented Emulation ($55,393, 106 units). The best day was 18 Sep 2026 ($129,397).

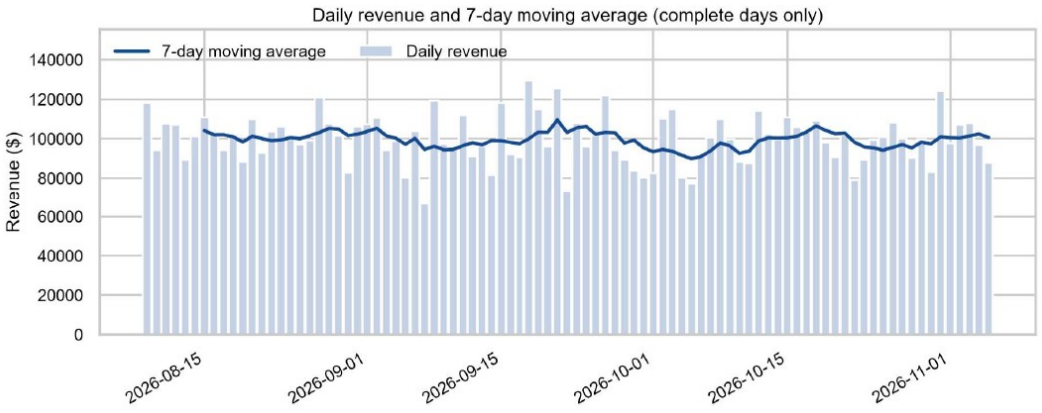


Figure 2 - Daily revenue (Spark SQL) with a 7-day moving average. I left out the partial first and last day of the window.

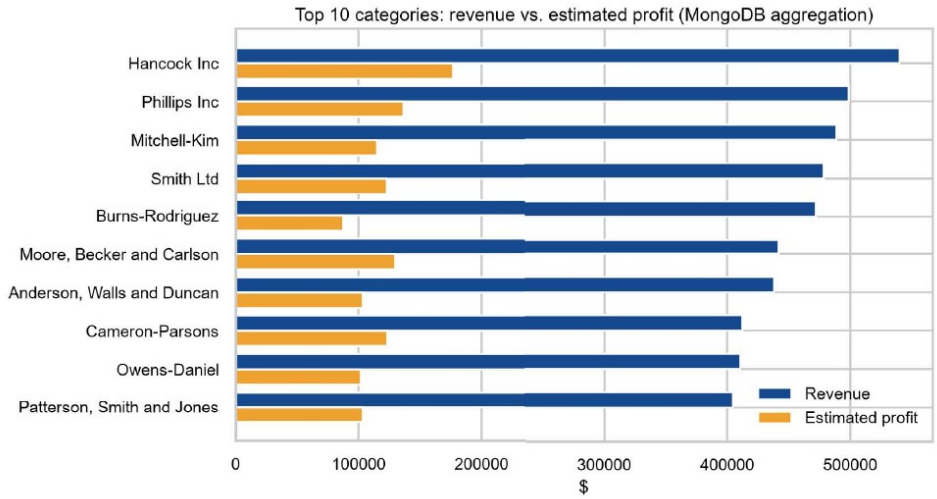


Figure 3 - Top 10 categories by revenue vs. estimated profit (MongoDB aggregation A). The profit ranking differs from the revenue ranking because margins differ by subcategory.

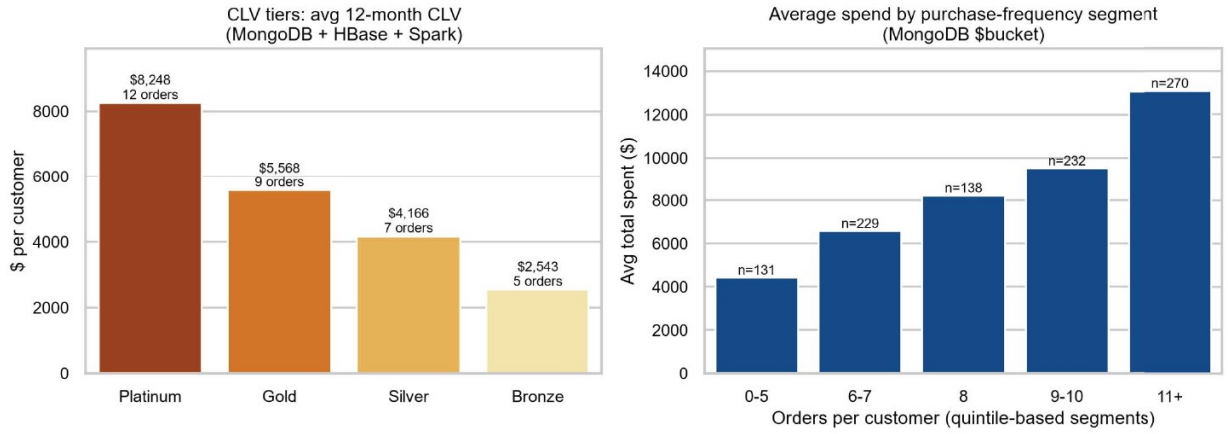


Figure 4 - Customer segmentation: CLV tiers (left) and purchase-frequency quintile segments (right).

Segmentation insight. Platinum customers (the top 25%) have an average 12-month CLV of $8,248 versus $2,543 for Bronze (3.2x), and they generate 40% of total projected CLV. CLV correlates with the session conversion rate (r = 0.38) and session count (r = 0.20), but hardly at all with time on site (r = −0.01). **Action:** put the effort into what turns visits into orders (offers, checkout friction) rather than raw time-on-site, and look after the Platinum tier with retention programmes.

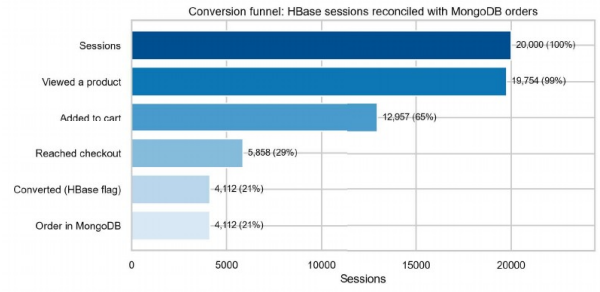


Figure 5 - Conversion funnel (HBase sessions reconciled with MongoDB orders).

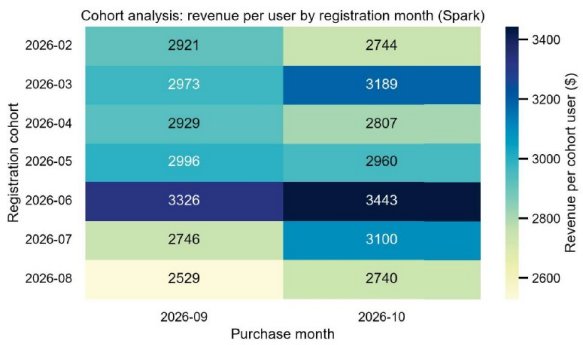


Figure 6 - Cohort analysis: revenue per user by registration and purchase month (Spark).

Funnel insight. 99% of sessions view a product, 65% add to cart, 29% reach checkout and 21% convert. The biggest loss is **added to cart → reached checkout** (a 55% drop). **Action:** prioritise fixes at that step, for example persistent carts and reminder e-mails for carts that reached checkout.

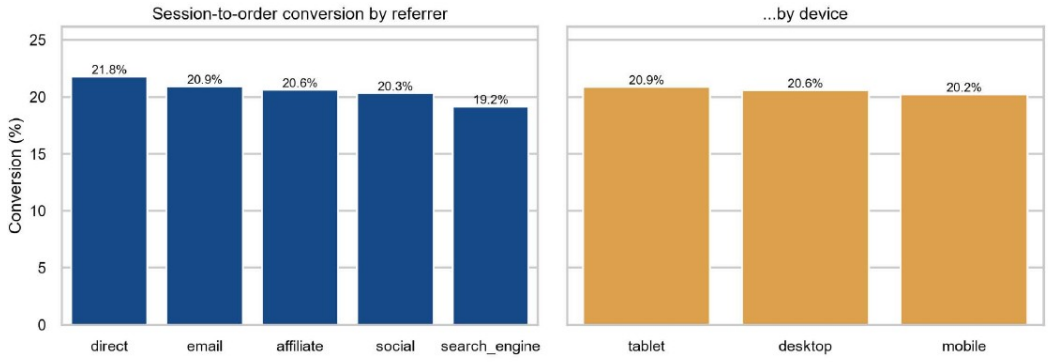


Figure 7 - Session-to-order conversion by referrer and by device.

Channel insight. Conversion by referrer ranges from 19.2% (search_engine) to 21.8% (direct), a spread of 2.6 percentage points. By device the spread is only 0.7 points, which is negligible. **Action:** review landing pages and bids for the weakest channel (search_engine) before changing any device-specific experiences.

Cohort insight. Revenue per user varies very little between cohorts and months (coefficient of variation 0.08), so there is no visible decay or growth with customer age. The data are synthetic and purchases are assigned at random, so please do not read these cohort patterns as real-world findings. Calendar months with fewer than 25 days of data (the partial months at the start or end of the 90-day window) are excluded.

6. Scalability considerations

- **MongoDB:** shard transactions on a hashed user_id (or a time bucket) so writes spread out; keep the user summaries to avoid a cross-shard $lookup; use compound indexes that match the pipelines. The full dataset (500k orders) fits on one replica set easily.
- **HBase:** it scales through region splits across RegionServers. The key leads with user_id (not the timestamp) to avoid hotspots; tables are pre-split; TTL and compaction retire old sessions; batch Puts reduce RPC overhead. At 2M sessions the loader should switch to bulk load (ImportTsv/HFiles) or the Spark-HBase connector.
- **Spark:** partitioned Parquet (the 4 GB of JSON shrinks several-fold), broadcast of small dimensions, tuned shuffle partitions and explicit schemas. The same code runs on YARN or Kubernetes just by changing the master. Avoid collecting large results to the driver.

7. Technology selection summary

Task	Technology	Why
Revenue, profit, top products, customer summaries	MongoDB	Flexible nested documents + aggregation pipeline + indexes.
User session timeline, product daily metrics	HBase	Time-ordered row keys, range scans, sparse wide rows.
Cleaning, recommendations, cohorts, SQL analytics	Spark	Distributed scans and joins over all sessions; SQL + DataFrames.
CLV and funnel	All three	Needs orders (Mongo) + engagement (HBase) + a join/compute engine (Spark).

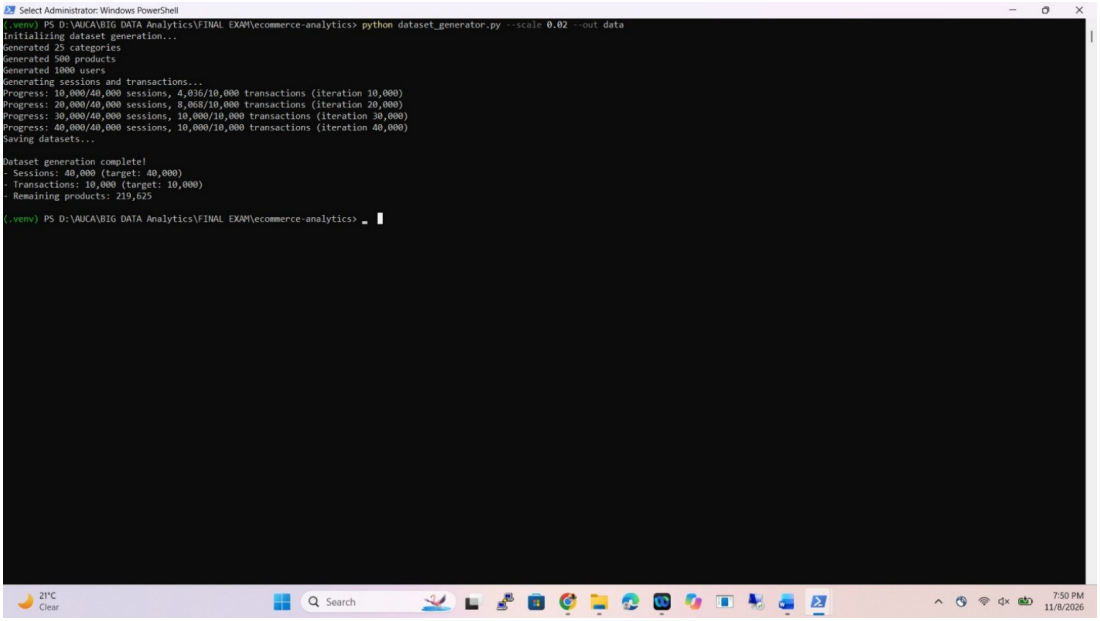
8. Limitations and future work

- **Synthetic data:** purchases and sessions are random, so cohorts, segments and correlations are weak or flat, and lift on small baskets is noisy. The findings demonstrate the method, not real customer behaviour.
- **Generator artefacts:** there is no subcategory_id on products (I assign one by rule), no age (so age-group segmentation is impossible), there are orphan transactions, and there are converted sessions without orders after the transaction cap.

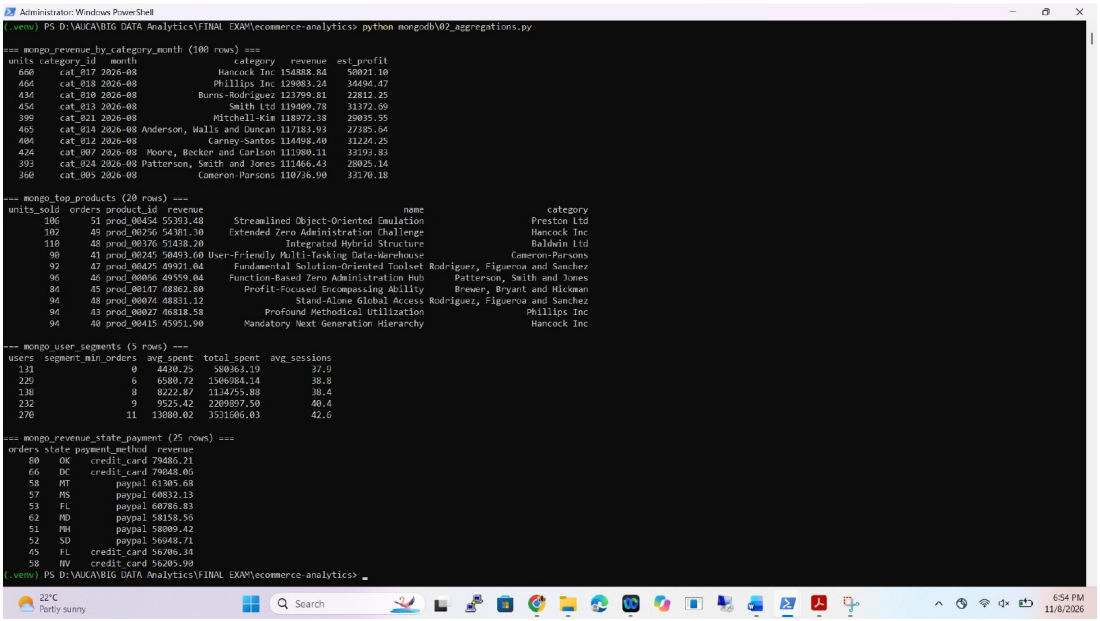
- **CLV:** it uses a fixed assumed retention (0.90) and a 90-day window, and there is no churn model. MongoDB holds all orders and users, but HBase was loaded with a subset of the sessions (the first session files). The integration reads through single-threaded Python clients, so the engagement figures come from that subset.
- **Future work:** ALS collaborative filtering with Spark MLlib; a Structured Streaming simulation of live sessions into HBase; MongoDB change streams to keep the user summaries fresh; RFM + K-means segmentation; Kafka ingestion; and connector-based parallel reads.

Appendix A - Console captures of the pipeline

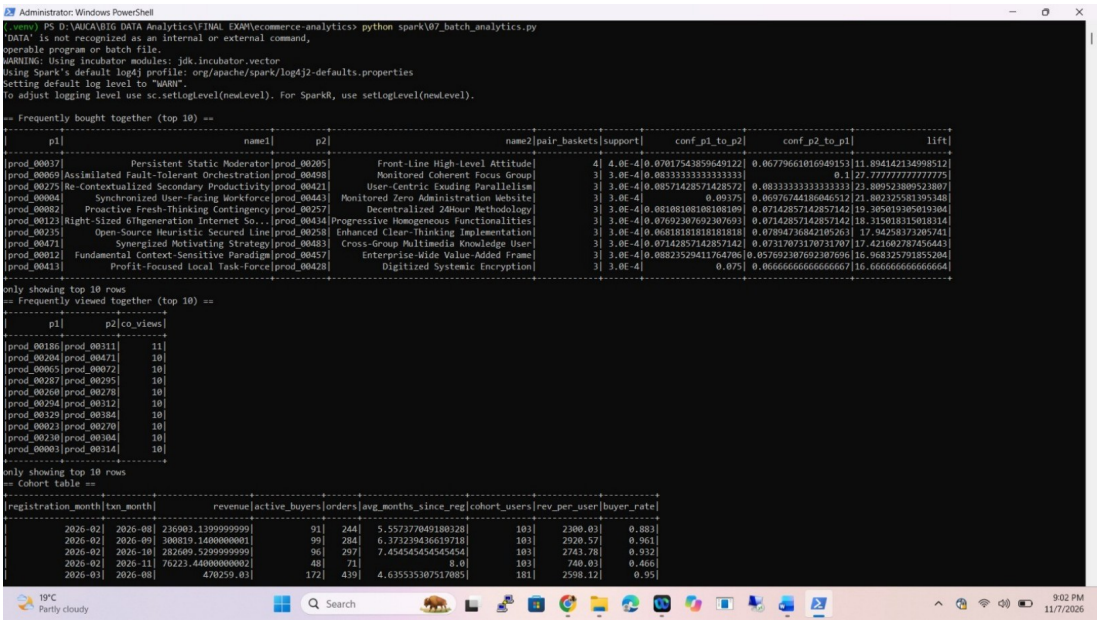
These are real screenshots from my own run of each pipeline step on Windows (PowerShell, Python 3.12, Java 17, PySpark 4.3 in local mode). MongoDB ran as a real server on the same machine (localhost:27017). I could not install HBase without Docker, so the HBase step and the integration job used the project's offline test harness (in-memory stand-ins) for the two stores. All of the Spark steps and the verification ran for real.



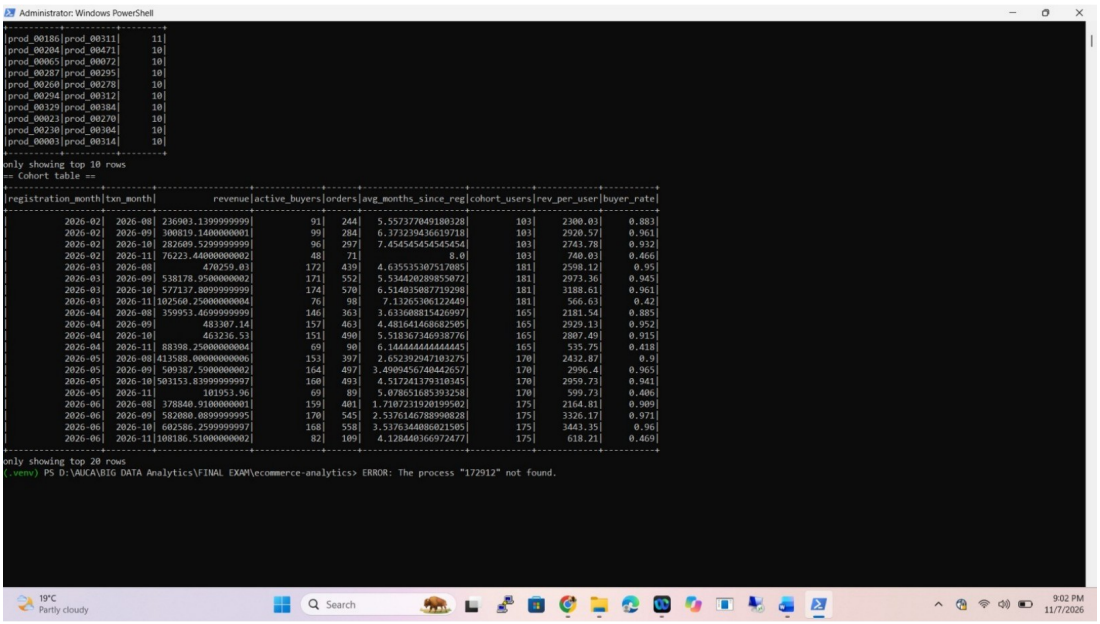
A1 - Dataset generation (dataset_generator.py --scale 0.02): 40,000 sessions and 10,000 transactions.



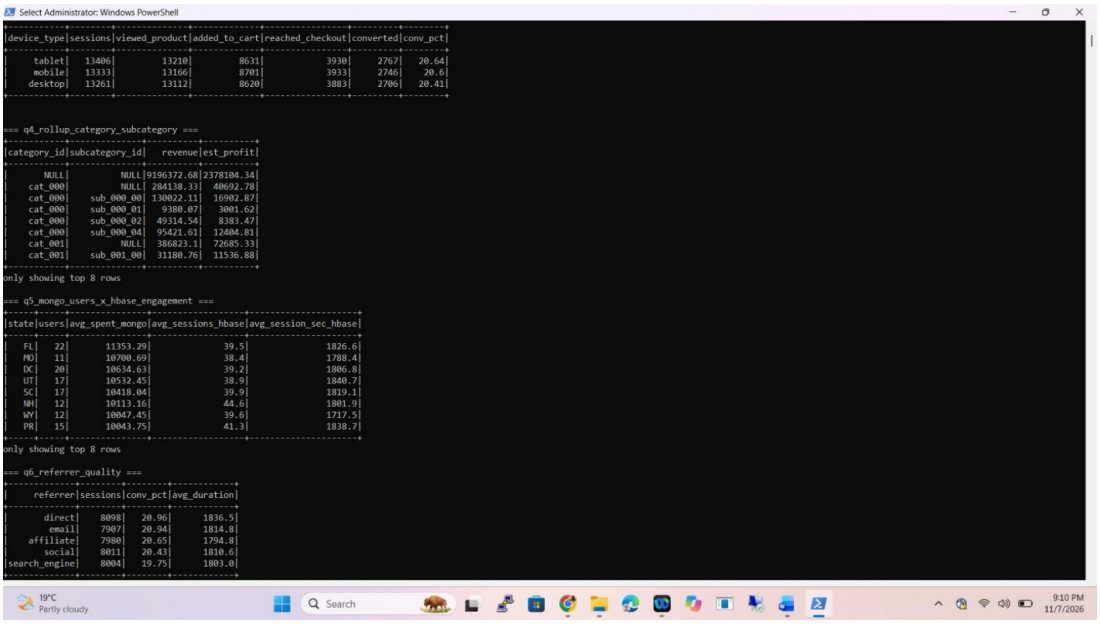
A2 - MongoDB pipelines A-D on the real MongoDB server, including B (top products) and C (segmentation by quintiles).



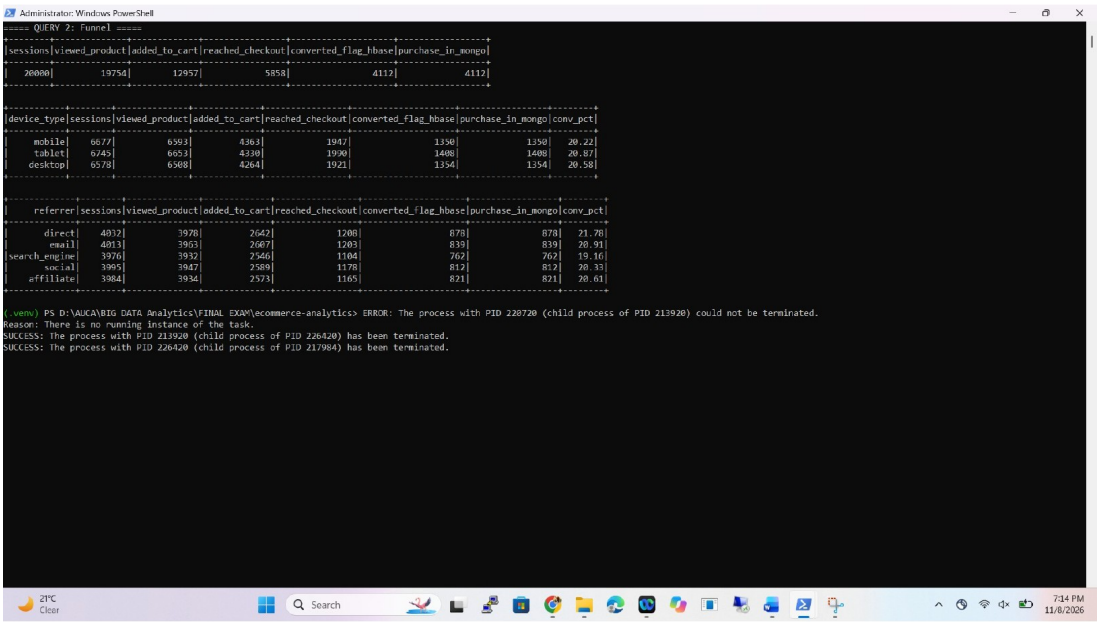
A3 – Spark: frequently bought together (support, confidence, lift) and frequently viewed together.



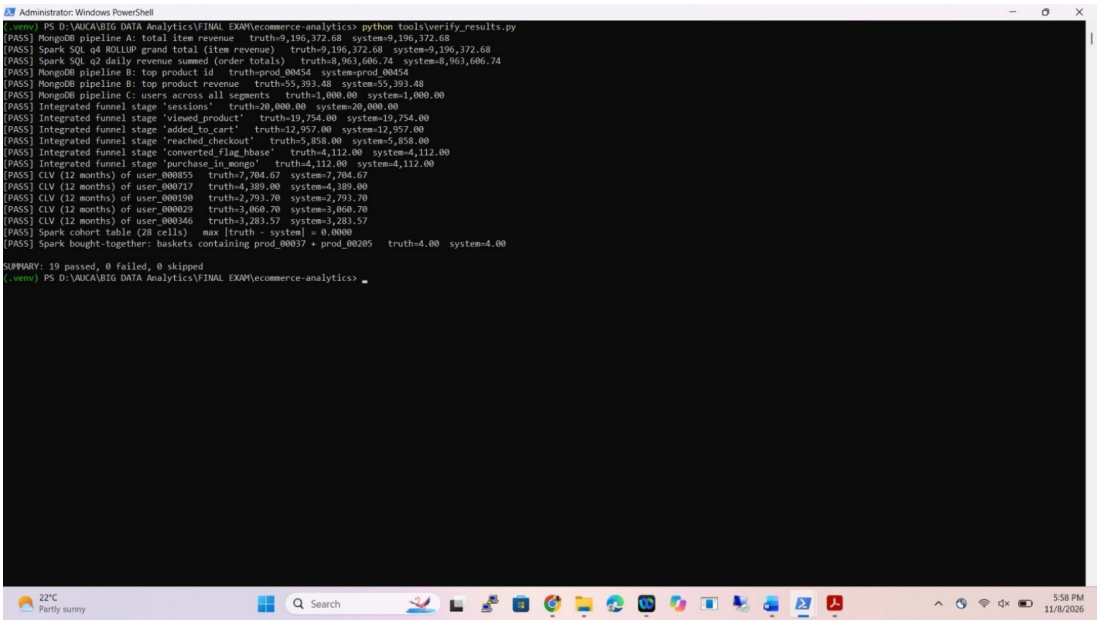
A4 – Spark: cohort table (revenue per user by registration month and purchase month).



A5 – Spark SQL: device funnel (q3), ROLLUP (q4), MongoDB x HBase engagement join (q5) and referrer quality (q6).



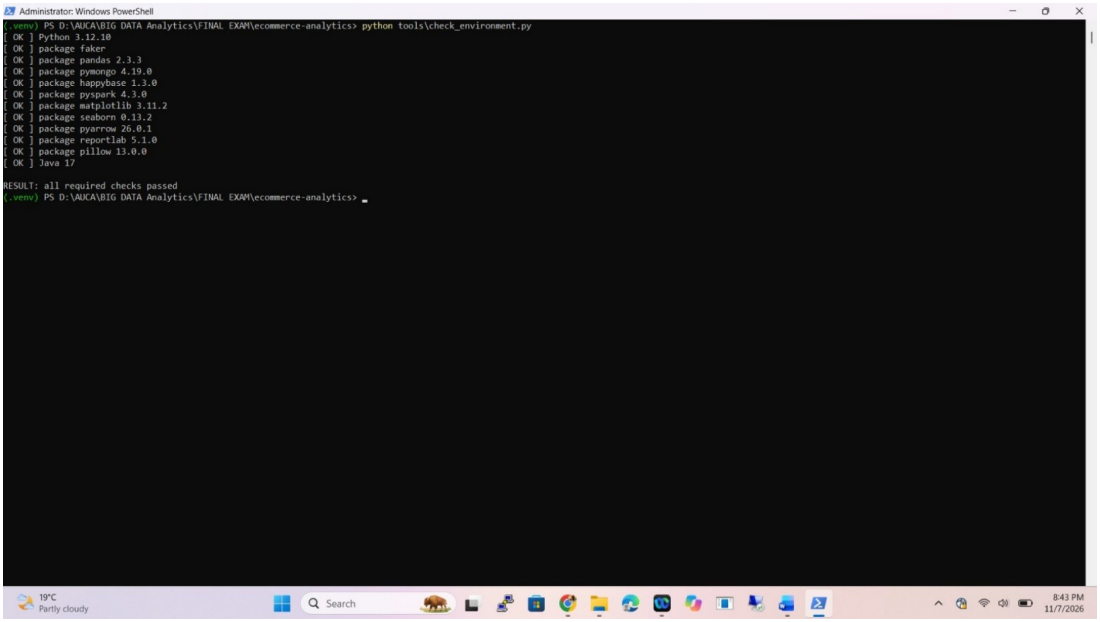
A6 – Integrated query 2: funnel (HBase sessions x MongoDB orders), overall, by device and by referrer. Offline test harness for the two stores.



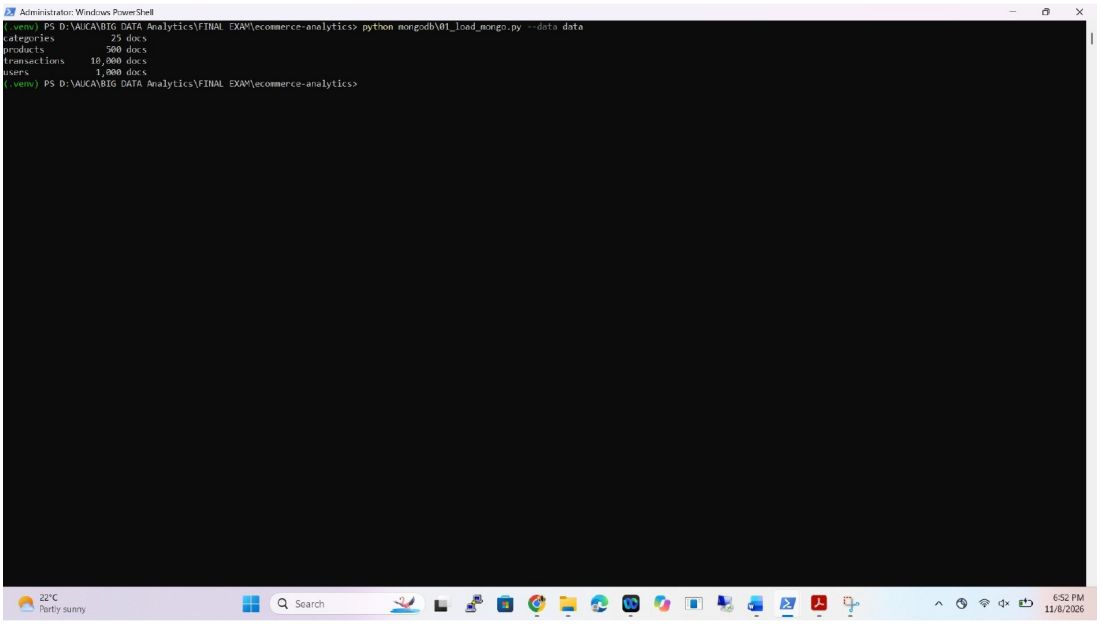
A7 - Independent verification of the results against the raw JSON (plain Python, no database or Spark): 19 passed, 0 failed.

Appendix B - Further screenshots from my own run

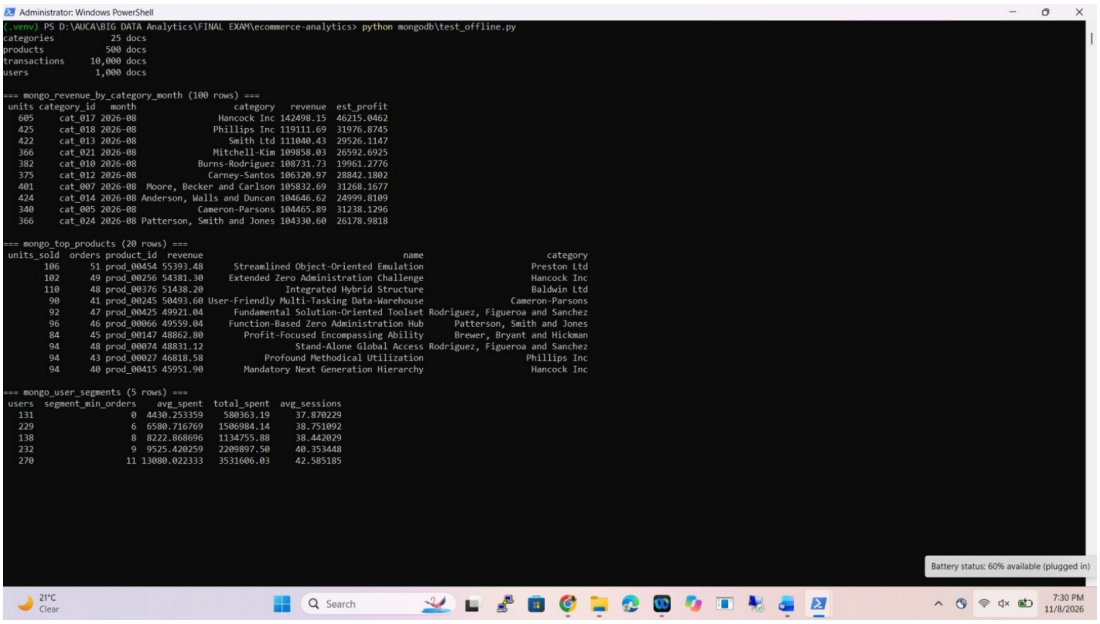
A few more real screenshots: the environment check, loading the real MongoDB server, the MongoDB and HBase offline tests, the Spark cleaning job and SQL queries 1-3, integrated query 1 (CLV), the visualisation step, and the Spark web UI during a Spark session.



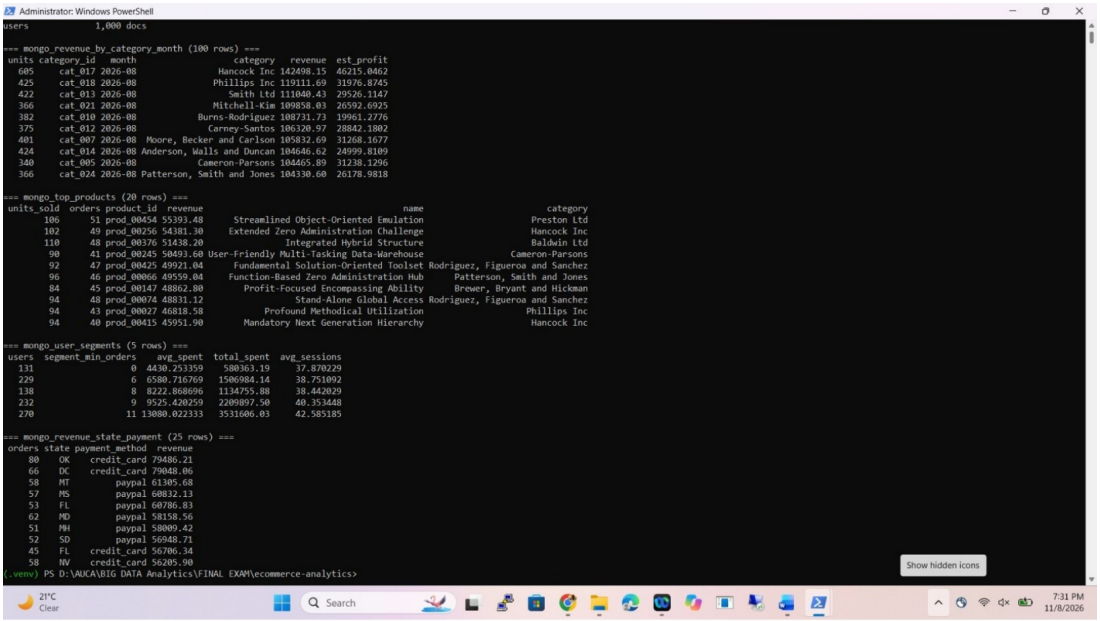
B1 - Environment check: Python 3.12, all packages and Java 17 installed.



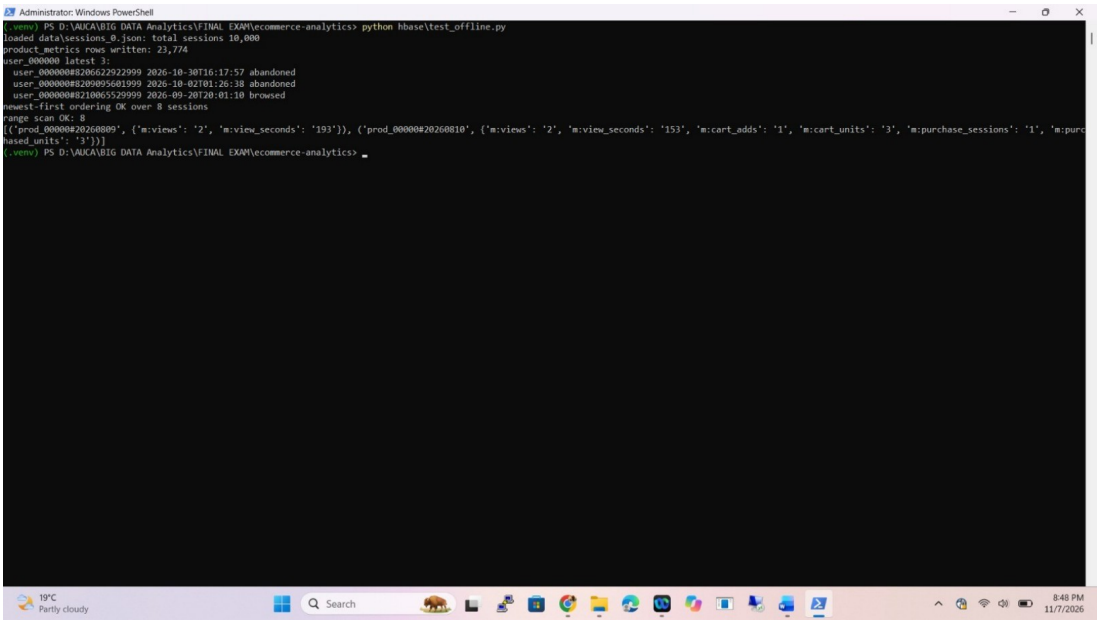
B2 - MongoDB (real server on localhost:27017): data loaded with mongodb/01_load_mongo.py - 25 categories, 500 products, 10,000 transactions, 1,000 users.



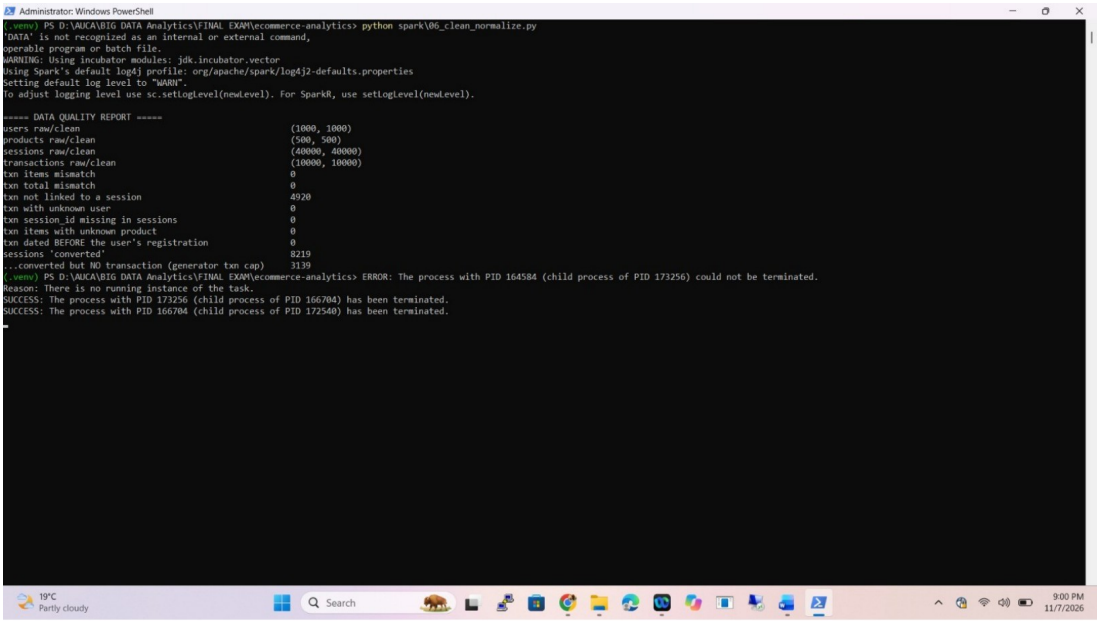
B3 – MongoDB offline test (mongodb/test_offline.py): pipeline A (revenue by category and month), B (top products) and C (segmentation). Offline test harness.



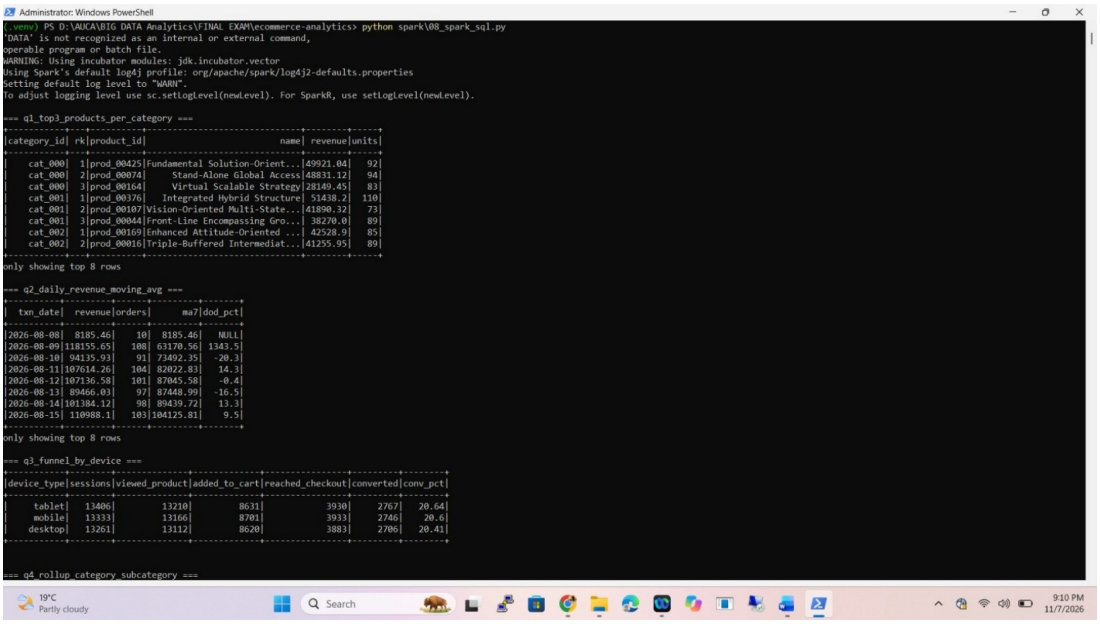
B4 – MongoDB offline test, continued: pipelines B, C and D (revenue by state and payment method). Offline test harness.



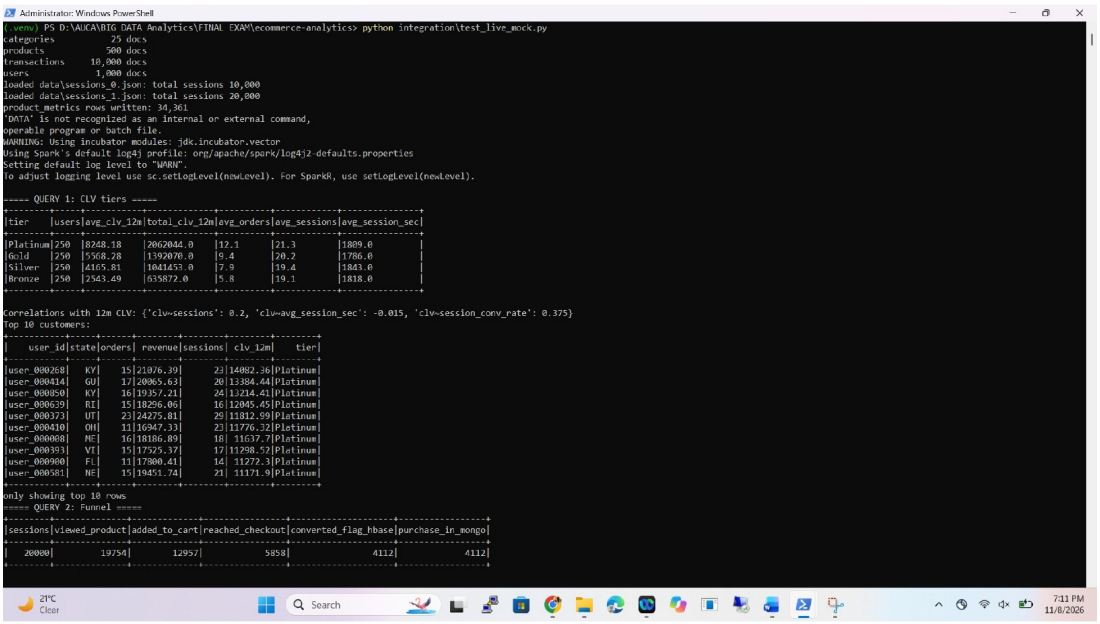
B5 – HBase: sessions loaded, newest-first scan of one user's sessions, range scan and product_metrics rows. Offline test harness.



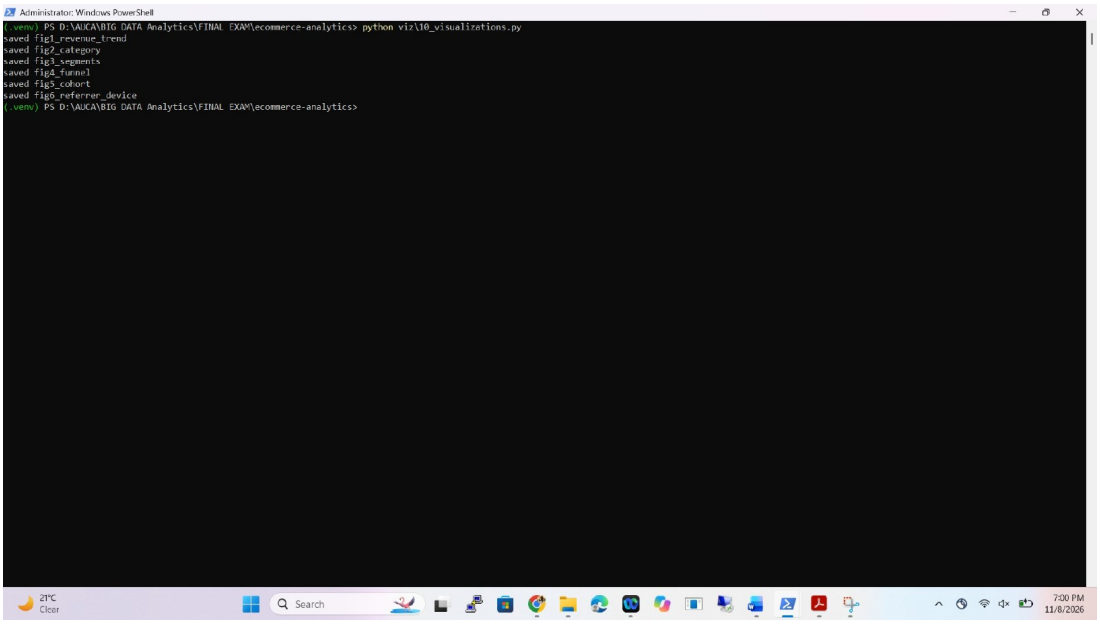
B6 – Spark cleaning and normalisation job with its data-quality report.



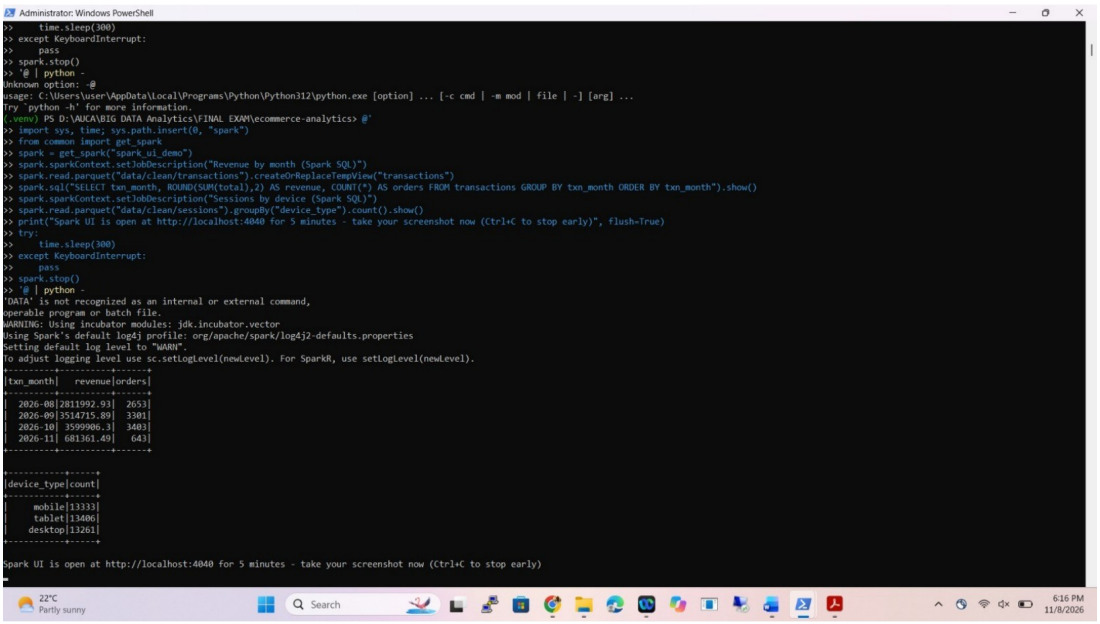
B7 – Spark SQL queries 1-3: top-3 products per category, 7-day moving average, funnel by device.



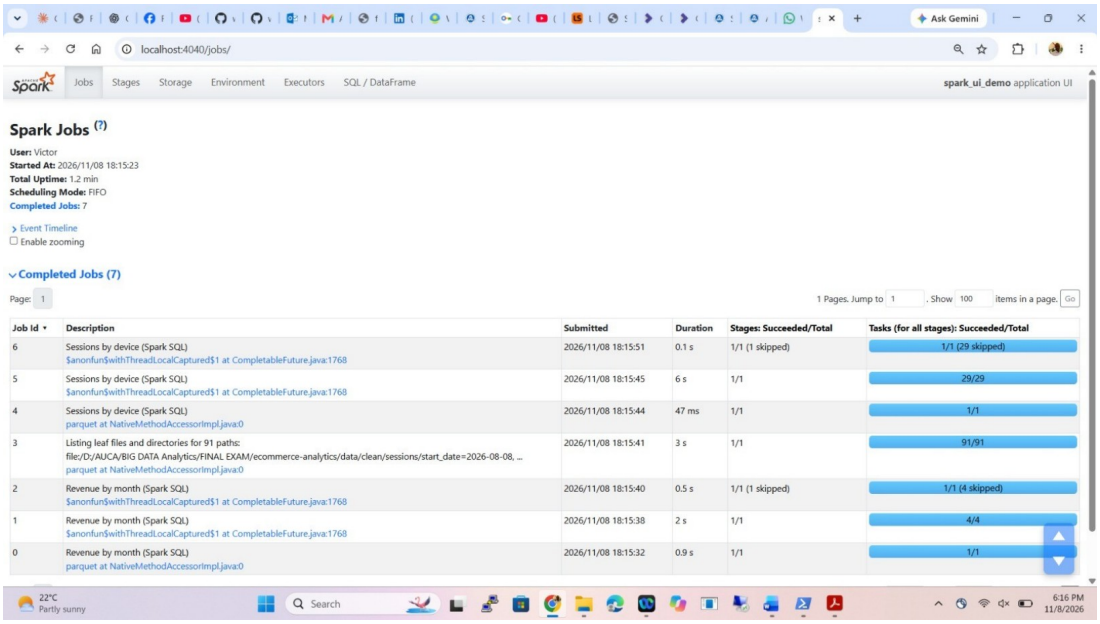
B8 – Integrated query 1: customer lifetime value tiers, correlations and top 10 customers (MongoDB + HBase + Spark). Offline test harness for the two stores.



B9 – Visualisations: viz/10_visualizations.py saving the six figures used in section 5.



B10 – Spark session kept open for the Spark web UI (revenue by month and sessions by device).



B11 – Spark web UI (http://localhost:4040), Jobs page: 7 completed jobs, all stages succeeded.